Actividad Núcleo 3: Despliegue de una aplicación con base de datos

LENGUAJES DE PROGRAMACION PARA APLICACIONES DE NUBE

JONATHAN DAVID RODRIGUEZ VELANDIA

2026

Requisitos Previos

- **JDK 17** o superior instalado y configurado en las variables de entorno (`JAVA_HOME`).
- **Apache Maven 3.8+** (o el wrapper integrado `./mvnw`).

⚙️ Instrucciones de Instalación y Ejecución

1. Clonar o Navegar al Directorio

Bash
```
cd banknode-atm
```

2. Ejecutar Pruebas Automatizadas

Bash
```
mvn clean test
```

3. Ejecutar en Modo Desarrollo (Spring Boot)

Bash
```
mvn spring-boot:run
```

4. Empaquetar y Ejecutar JAR Autónomo (Producción)

Bash
```
# Compilar el Fat JAR
mvn clean package -DskipTests

# Ejecutar el artefacto compilado
java -jar target/banknode-atm-1.0.0.jar
```

🔑 Credenciales y Datos de Prueba

Al iniciar la aplicación, la clase `BanknodeAtmApplication.java` inicializa automáticamente la base de datos con los siguientes clientes y el inventario del cajero:

Clientes Precargados:

Documento	PIN	Nombre	Saldo Inicial

1001	1234	Carlos Mendoza	$1.500.000 COP
1002	4321	Laura Gómez	$850.000 COP
1003	9876	Andrés Restrepo	$2.300.000 COP

Inventario Inicial del Cajero:

- **$100.000:** 40 Billetes ($4.000.000)
- **$50.000:** 50 Billetes ($2.500.000)
- **$20.000:** 50 Billetes ($1.000.000)
- **$10.000:** 100 Billetes ($1.000.000)
- **Total en Efectivo Disponible: $8.500.000 COP**

🌐 Puntos de Acceso Web

- **Pantalla 1 (Disponibilidad y Arqueo):** http://localhost:8080/
- **Pantalla 2 (Login de Cliente):** http://localhost:8080/login
- **Pantalla 3 (Operaciones / Dashboard):** Accesible tras autenticación exitosa.
- **Consola H2 Database:** http://localhost:8080/h2-console
  - *JDBC URL:* `jdbc:h2:mem:banknodedb`
  - *User:* `sa`
  - *Password: (En blanco)*

🛡 Casos de Uso y Reglas de Negocio

1. **Pantalla 1 - Disponibilidad de Billetes:**

Muestra el saldo global del cajero y la discriminación por denominación. Si alguna denominación se agota, muestra una advertencia indicando qué valor hace falta.

2. **Pantalla 2 - Autenticación y Bloqueo:**

Valida el número de documento y PIN de 4 dígitos.

   a. Cada intento fallido descuenta los intentos restantes.
   b. Al **tercer intento fallido**, la cuenta entra en estado bloqueado por **10 minutos** exactos.
   c. Si el usuario intenta ingresar durante el bloqueo, el sistema muestra los minutos faltantes. Un ingreso exitoso restablece el contador de intentos.

3. **Pantalla 3 - Operaciones:**

    a. **Consulta de Saldo:** Muestra el saldo actualizado de la cuenta en tiempo real.
    b. **Retiros:** Valida que el monto sea múltiplo de $10.000, existan fondos suficientes en la cuenta y billetes disponibles en la máquina. Aplica un algoritmo codicioso para despachar primero los billetes de mayor denominación.
    c. **Transferencias:** Permite enviar dinero a otros clientes registrados debitando y acreditando los fondos de forma atómica.
4. **Cierre de Sesión por Inactividad (180 segundos):**

Un script en JavaScript monitoriza los eventos del ratón y teclado. Si transcurren 180 segundos sin interacción, destruye la sesión y redirige al inicio informando la expiración de la sesión.

🧪 Pruebas Automatizadas

El proyecto incluye 7 pruebas de integración y unitarias que validan la lógica central (`BanknodeAtmApplicationTests.java`):

- `contextLoadsYDatosIniciales`: Verifica la carga del contexto y el sembrado de datos iniciales.
- `testOperacionesEntidadCliente`: Comprueba los métodos de consulta, consignación y retiro en la entidad.
- `testGestionInventarioCajero`: Revisa el desglose de billetes y validación de múltiplos.
- `testAutenticacionExitosa`: Valida el ingreso correcto y reinicio de contadores.
- `testBloqueoPorTresIntentosFallidos`: Garantiza el bloqueo por 10 minutos tras 3 fallos.
- `testTransferenciaEntreCuentas`: Asegura que el débito y crédito entre cuentas sea preciso.
- `testRetiroCompletoCajeroService`: Prueba la transacción integral entre cuenta bancaria e inventario físico de billetes.